

Métodos de Otimização em Contexto Industrial

Otimização não diferenciável sem restrições

MATLAB - Comando `fminsearch`

Ana Maria A. C. Rocha

Departamento de Produção e Sistemas

Universidade do Minho

arocha@dps.uminho.pt

Ano letivo 2023/24

MATLAB - Comando `fminsearch`

Comando `fminsearch`

- mínimo de problemas de otimização não diferenciáveis sem restrições
- mínimo de uma função multivariável sem restrições
- variáveis de números reais
- usa o método de Nelder-Mead de procura direta.

Para ver a estrutura das opções de otimização disponíveis:

```
optimset('fminsearch')
```

Sintaxe de utilização

```
[x,fval,exitflag,output] = fminsearch(fun,x0,opt,...)
```

Sintaxe de utilização

```
[x,fval,exitflag,output] = fminsearch(fun,x0,opt,...)
```

Argumentos de entrada:

fun - é a função objetivo a minimizar

x0 - é a aproximação inicial

opt - opções de controlo do processo de otimização.

- Especificar **fun** como um identificador de função para um ficheiro:

```
x = fminsearch(@fun,x0)
```

em que o ficheiro **fun** é uma função MATLAB® definida por

```
function f = fun(x)
```

```
    f = ... ;
```

```
end
```

- Especificar **fun** como um identificador de função anónima, p.ex.:

```
x = fminsearch(@(x)...,x0)
```

Sintaxe de utilização: opt - opções de controlo

Display	Nível de apresentação. notify - (default) apresenta resultado se a função não co final - apresenta apenas o resultado final off - não apresenta nada iter - apresenta resultados em cada iteração
MaxFunEvals	Nº máximo de cálculos da função (default=200*NumVar)
MaxIter	Nº máximo de iterações (default=200*NumVar)
PlotFcns	Representa graficamente medidas do progresso do algoritmo, ao longo das iterações. @optimplotx - aproximação à solução x. @optimplotfunccount - número de cálculos da função. @optimplotfval - valor da função objetivo.
TolFun	Tolerância de paragem da função objetivo (default=1e-4)
TolX	Tolerância de paragem em x (default=1e-4)
FunValCheck	Verifica se os valores da função objetivo são válidos. on - apresenta erro off - (default) não apresenta qualquer erro

Sintaxe de utilização

```
[x,fval,exitflag,output] = fminsearch(fun,x0,opt,...)
```

Argumentos de saída:

x - é a solução do problema.

fval - é o valor da função objetivo em x.

exitflag - descreve valores de saída do fminsearch.

output - estrutura com informação acerca do processo de otimização

exitflag

1	indica que a função convergiu para a solução x
0	indica que excedeu o MaxIter ou o MaxFunEvals.
-1	indica que a função não convergiu para uma solução

output

iterations	indica o número de iterações realizadas
funcCount	indica o número de cálculos da função
algorithm	indica o algoritmo usado
message	mensagem de saída

Exercício 1

Determine a solução da seguinte função não diferenciável, usando o comando `fminsearch`. Inicie o processo com $x^{(1)} = (1, 1)^T$.

$$\min f(x_1, x_2) = \max(|x_1|, |x_2 - 1|)$$

- 1 Sem usar qualquer opção.
- 2 Visualizando os resultados obtidos em cada iteração.
- 3 Visualizando graficamente os valores da função ao longo das iterações.
- 4 Visualizando os resultados por iteração e a representação gráfica dos valores da função objetivo ao longo das iterações.
- 5 Usando como tolerância de paragem $TolX = 10^{-10}$.
- 6 Usando como tolerância de paragem $TolFun = 10^{-12}$.
- 7 Usando como tolerância de paragem 20 iterações.
- 8 Usando como tolerâncias de paragem $TolX = 10^{-6}$, $TolFun = 10^{-2}$ e 100 como máximo de iterações.

Resolução: Exercício 1

Representação gráfica da função: `ezsurf('max(abs(x),abs(y-1))')`

- 1 Sem usar qualquer opção.

```
[x,fval,exitflag,output]=fminsearch(@NM1,[1,1])  
%funcao  
function [ f ] = NM1( x )  
    f=max(abs(x(1)),abs(x(2)-1));  
end
```

ou

```
x0=[1,1];  
[x,fval,exitflag,output]=fminsearch(@(x)max(abs(x(1)),abs(x(2)-1)),x0)
```

- 2 Visualizando os resultados obtidos em cada iteração.

```
op=optimset('Display','iter');  
[x,fval,exitflag,output]=fminsearch(@NM1,[1,1],op)  
%funcao ...
```

- 3 Visualizando graficamente os valores da função ao longo das iterações.

```
op=optimset('PlotFcns',@optimplotfval);  
[x,fval,exitflag,output]=fminsearch(@NM1,[1,1],op)
```

Resolução: Exercício 1 (cont.)

- 4 Visualizando os resultados por iteração e a representação gráfica dos valores da função objetivo ao longo das iterações.

```
op=optimset('Display','iter','PlotFcns',@optimplotfval);  
...
```

- 5 Usando como tolerância de paragem $TolX = 10^{-10}$.

```
op=optimset('TolX',1e-10);  
...
```

- 6 Usando como tolerância de paragem $TolFun = 10^{-12}$.

```
op=optimset('TolFun',1e-12);  
...
```

- 7 Usando como tolerância de paragem 20 iterações.

```
op=optimset('MaxIter',20);  
...
```

- 8 Usando como tolerâncias de paragem $TolX = 10^{-6}$, $TolFun = 10^{-2}$ e 100 MaxIter.

```
op=optimset('TolX',1e-6,'TolFun',1e-2,'MaxIter',100);  
...
```


Exercício 2

Determine a solução da seguinte função não diferenciável, usando o comando `fminsearch`. Inicie o processo com $x^{(1)} = (-1, 1)^T$.

$$\max f(x_1, x_2) = -|x_1 x_2| - x_2^2$$

- 1 Sem usar qualquer opção.
- 2 Usando como tolerância de paragem $TolX = 10^{-8}$.
- 3 Usando como tolerância de paragem $TolFun = 10^{-6}$ e visualizando graficamente os valores da função objetivo ao longo das iterações.
- 4 Usando o máximo de 50 iterações como tolerância de paragem.

Resolução: Exercício 2

Representação gráfica da função: `ezsurf(' -abs(x*y)-y^2')`

- 1 Sem usar qualquer opção.

```
[x,fval,exitflag,output]=fminsearch(@NM1,[-1 1])  
function [ f ] = NM1( x )  
    f=abs(x(1)*x(2))+x(2)^2;  
end
```

- 2 Usando como tolerância de paragem $TolX = 10^{-8}$.

```
op=optimset('TolX',1e-8);  
[x,fval,exitflag,output]=fminsearch(@NM1,[-1 1],op)  
...
```

- 3 Usando como tolerância de paragem $TolFun = 10^{-6}$ e visualizando graficamente os valores da função objetivo ao longo das iterações.

```
op=optimset('TolFun',1e-6,'PlotFcns',@optimplotfval);  
...
```

- 4 Usando o máximo de 50 iterações como tolerância de paragem.

```
op=optimset('Maxiter',50);  
...
```

Exercício 3

Determine a solução da seguinte função não diferenciável, usando o comando `fminsearch`. Inicie o processo com $x^{(1)} = (1, 0)^T$.

$$\min f(x_1, x_2) = \max((x_1 - 1)^2, x_2^2 + x_1, 4(x_2 - 1)^2)$$

- 1 Sem usar qualquer opção.
- 2 Use como tolerâncias de paragem $TolX = 10^{-6}$, $TolFun = 10^{-8}$
- 3 Use como tolerâncias de paragem o máximo 100 iterações, $TolX = 10^{-6}$, $TolFun = 10^{-8}$ e visualizando graficamente os valores da função objetivo ao longo das iterações.

Resolução: Exercício 3

Representação gráfica da função: `ezsurf('max([(x-1)^2,y^2+x,4*(y-1)^2])')`

- 1 Sem usar qualquer opção.

```
[x,fval,exitflag,output]=fminsearch(@NM1,[1, 0])  
function [ f ] = NM1( x )  
    u=[(x(1)-1)^2 , x(2)^2+x(1) , 4*(x(2)-1)^2];  
    f=max(u);  
end
```

- 2 Use como tolerâncias de paragem $TolX = 10^{-6}$, $TolFun = 10^{-8}$ e 100 como máximo de iterações.

```
op=optimset('TolX',1e-6,'TolFun',1e-8,'maxiter',100);  
[x,fval,exitflag,output]=fminsearch(@NM1,[1, 0],op)  
...
```

- 3 Use como tolerâncias de paragem $TolX = 10^{-6}$, $TolFun = 10^{-8}$ e visualizando graficamente os valores da função objetivo ao longo das iterações.

```
op=optimset('TolX',1e-6,'TolFun',1e-8,'PlotFcns',@optimplotfunccount))  
[x,fval,exitflag,output]=fminsearch(@NM1,[1, 0],op)  
...
```